

Proyecto ToDo List

Realizado por Sergio Rodríguez Geniz

Descripción

Este proyecto es una aplicación de consola ejecutada en Python que permite la gestión de tareas mediante un CRUD (Creación, Listado, Edición y Borrado) de las mismas. Las tareas creadas se almacenan en un archivo en el dispositivo en formato JSON lo que permite el guardado de la información en diferentes ejecuciones.

Objetivo del proyecto

El objetivo principal de este proyecto fue la práctica de programación en el lenguaje Python, utilizando diferentes estructuras de control, importando librerías nuevas como JSON, llevando a cabo la apertura y cierre de ficheros para llevar a cabo persistencia de datos entre diferentes ejecuciones, utilizando plantillas que nos permite la Programación Orientada a Objetos como son las clases y por último nos ayuda a estructurar el proyecto en diferentes módulos para poder crear código más seguro y escalable.

Funcionalidades del proyecto

1. Añadir tareas: El programa permite al usuario añadir nuevas tareas para poder complirlas en un futuro o introducirlas completadas.
2. Eliminar tareas: La aplicación permite al usuario eliminar tareas creadas anteriormente que ya se hayan completado o que estén por completar.
3. Editar tareas: La aplicación permite al usuario editar todos los datos (el nombre de la tarea, la descripción, las categorías, la prioridad y si está completada) de las tareas creadas anteriormente.
4. Listar tareas: El programa permite que el usuario vea todos los datos de las tareas en forma de listado, indicando el ID de cada tarea para su futura utilización.
5. Generar reportes: La aplicación permite crear un reporte al usuario haciendo un recuento de las tareas completadas, las no completadas y el porcentaje de tareas que lleva el usuario completadas.
6. Marcar tareas como completadas: El programa permite cambiar el estado de las tareas no completadas cambiando el valor del campo 'completada' de Falso a Verdadero. Si se selecciona una tarea ya completada anteriormente, al indicar el ID de la tarea se mostrará un mensaje de error indicando que la tarea ya fue completada anteriormente.
7. Buscar tareas: La aplicación permite que el usuario realice búsquedas de tareas de dos maneras diferentes:

- a. Categorías: El programa le pedirá al usuario una categoría a buscar y mostrará todas las tareas que tengan la categoría indicada por el usuario en cualquier posición.
 - b. Prioridad: El programa le solicitará al usuario la prioridad por la que desea buscar las tareas y le mostrará una lista ordenada de todas las tareas que tienen la prioridad indicada.
8. Persistencia de datos: El programa permite guardar el listado de tareas en un archivo JSON para guardar las tareas en diferentes ejecuciones. Al iniciar la aplicación, está lee el archivo JSON y obtiene las tareas a utilizar.

Estructura del proyecto

El proyecto está compuesto por un total de 6 archivos.

- **main.py**: Este es el archivo principal, el cual controla el menú interactivo del usuario
- **data_handler.py**: Este archivo controla el flujo de datos en la aplicación, es decir, lleva a cabo la apertura y el cierre del archivo JSON y añade, elimina y busca tareas creadas anteriormente.
- **utils.py**: Este archivo lleva a cabo las comprobaciones de todos los datos insertados, del listado de tareas y de la conversión de datos leídos desde el archivo JSON.
- **reports.py**: Este archivo realiza un reporte sobre las tareas creadas, indicando el número total de tareas existentes, el número de tareas completadas, el número de tareas no completadas y el porcentaje que tenemos de completado en total.
- **Tarea.py**: Este archivo permite crear el formato de las tareas, indicando los campos que las componen.
- **tareas.json**: Este es el archivo en el que se almacenan todas las tareas creadas. El archivo es leído al iniciar la aplicación si existe, si no, se creará.

Estructura de las tareas

Las tareas se componen de diferentes atributos variables.

- **nombreTarea**: Es de tipo String (str) y guarda el nombre de la tarea.
- **descripcion**: Es de tipo String (str) y guarda una descripción de la tarea.
- **categoria**: Es de tipo diccionario (str, str) en el que la clave es un número (incrementado de manera automática y parseado a tipo str) y el valor es una cadena de texto. Indica la categoría en la que está incluida la tarea.
- **prioridad**: Es de tipo Integer (int). Indica la prioridad de la tarea. Esta puede estar entre 3 valores predeterminados.
 - **Valor 1**: Indica que la tarea es urgente.
 - **Valor 2**: Indica que la tarea tiene una prioridad media.

- **Valor 3:** Indica que la tarea tiene una prioridad baja.
- **completada:** Es de tipo boolean (bool) e indica si la tarea está completada o no.

Estructura del archivo JSON

El archivo JSON es el que permite la persistencia de datos. Este archivo tiene el siguiente formato:

- **{}**: Las llaves de apertura y cierre contienen toda la información de las tareas.
- **"n"**: Indican el ID de cada tarea. Este ID debe de ser único.
- **{}**: Las llaves guardan los datos de la tarea.
- **"campo"**: Indican el campo del que guardar datos.
- **":"**: Permiten añadir el valor del campo.
- **"Valor"**: Indican los datos que guardamos en cada campo.

Ejemplo del formato JSON

```
{
  "1" : {
    "nombreTarea": "nombreTarea",
    "descripcion": "descripcion",
    "categoria" : {
      "1" : "categoria",
      "2" : "Cat2"
    },
    "prioridad" : 3,
    "completada" : True
  },
  "2" : {
    "nombreTarea" : "nombreTarea 2",
    "descripcion" : "descripcion 2",
    "categoria" : {},
    "prioridad" : 2,
    "completada" : false
  }
}
```

Requisitos del sistema

Para poder ejecutar la aplicación sin problemas algunos deberás tener instalado una versión de **Python** igual a la **3.13.8** o superior.

Las librerías utilizadas para este proyecto fueron:

- **JSON:** Para la lectura y guardado de datos.
- **OS:** Para la búsqueda, comprobación de existencia del fichero y creación del mismo.

Problemas encontrados y soluciones

- **KeyError:** Este error se produce cuando se intenta acceder a un índice del diccionario y este índice no existe.
Este error se producía cuando al crear nuevas tareas se intentaba editar o eliminar.
Solución: Tras ejecutar el programa en modo desarrollador, logré observar que al crear una nueva tarea el ID se guardaba como un número entero mientras que el ID de las tareas leídas desde el JSON era de tipo String. Esto se solucionó unificando todos los datos para que tuvieran el mismo tipo y a su vez se comprobaba mediante una conversión que el ID fuera del mismo tipo y existiera en la lista.
- **JSONDecodeError:** Este error se producía al iniciar la aplicación por primera vez cuando no existía el archivo JSON o tras un error provocando que el archivo quedara vacío o corrupto.
Solución: Al iniciar la app comprobamos que el archivo existe, y si no existe se crea vacío. Si el archivo existe intentamos leerlo e intentar convertir los datos a formato tarea. Si el archivo se encontraba vacío o corrupto saltaba una excepción, por lo que comprobamos que el archivo se puede leer. Si no es posible leer datos del archivo se creará uno nuevo vacío.

Diagrama de flujo

El siguiente diagrama de flujo representa el funcionamiento de la aplicación ToDo List.

Como se puede observar en el inicio del programa abrimos el archivo JSON y lo leemos (Si no existe se creará un archivo en blanco con un diccionario vacío).

Seguidamente solicitamos al usuario que introduzca una opción por teclado y dependiendo la opción introducida realizamos acciones diferentes.

- Si introducimos el valor "0" el programa realizará un guardado de tareas en el archivo JSON y se finalizará la ejecución del mismo.
- Si se introduce el valor "1", el programa se dirigirá al método listarTareas (se encuentra en el archivo utils.py) que mostrará al usuario todas las tareas creadas.
- Si se introduce el valor "2", el programa obtendrá un ID no existente para añadir tareas, y seguidamente se llamará al método addTarea (en el archivo data_handler.py) el cual le solicitará al usuario los datos necesarios para la

creación de una nueva tarea. Los datos añadidos por el usuario se comprobarán y si estos son correctos se añadirán a la tarea, si no se le solicitará de nuevo el dato al usuario.

- Si se introduce el valor “3”, se le solicitará al usuario el ID de la tarea que desee editar. El ID introducido deberá existir, si no se le solicitará uno nuevo. Una vez el usuario haya introducido el ID correctamente, nos dirigiremos al método `editarTareas` (en el archivo `data_handler.py`) y se le solicitará al usuario un nuevo id para elegir la opción que desea editar, es decir, el nombre de la tarea, la descripción, etc.... Si el usuario desea editar las categorías, nos dirigiremos al método `editCategorias` (localizado en el mismo archivo) el cual le mostrará al usuario un ID junto a las categorías que tiene la tarea y le pedirá nuevamente el id para modificar ese ítem. Si el usuario introduce el valor “0” se saldrá primeramente del método `editCategorias` (si se encuentra dentro de este) y seguido deberá volver a introducir el valor “0” para salir del método de edición de tareas.
- Si se introduce el valor “4”, se le solicitará al usuario un ID, y se comprobará si el ID introducido existe, si no se le solicitará uno nuevamente. Este nos dirigirá al método `eliminarTarea` (en el archivo `data_handler.py`) y este comprobará que el ID introducido se encuentra en la lista de tareas y eliminará la tarea si existe. Si no existe el ID mostrará un mensaje de tarea inexistente.
- Si se introduce el valor “5”, se le generará un reporte al usuario indicando el número de tareas completadas, las no completadas y el porcentaje de completado que lleva. Este reporte se realiza desde el método `getTareasCompletadas` (se encuentra en el archivo `reports.py` y es un método recursivo) y devolverá un número con las tareas completadas.
- Si se introduce el valor “6”, se le solicitará al usuario un ID de una tarea existente. Si este ID no existe se le solicitará uno nuevo. Este apartado permite al usuario marcar como completada una tarea que estuviera en estado no completado. Si la tarea se encuentra en estado completado se mostrará un mensaje indicando que ya fue completada anteriormente.
- Si se introduce el valor “7”, se permitirá al usuario la búsqueda de tareas mediante categorías o prioridad. Al entrar en este apartado se le solicitará al usuario una opción, si el usuario introduce la opción de categorías se le pedirá al usuario la categoría por la que desea buscar. Si el usuario introduce la opción para la búsqueda de tareas mediante prioridad se le solicitará la prioridad a buscar. El método que permite tanto la búsqueda por categorías como por prioridad es el método `buscarTarea` (se encuentra en el archivo `data_handler.py`) y devuelve un diccionario con las tareas encontradas que se corresponden con la búsqueda elegida por el usuario.

Arquitectura

La arquitectura del proyecto sigue una estructura modular y organizada, diseñada para separar la lógica de la gestión de datos y de las comprobaciones pertinentes. El flujo principal de la aplicación se encuentra en el archivo `main.py`, mientras que los otros archivos se encargan de las comprobaciones pertinentes (como es el archivo `utils.py`), el tratamiento de datos (`data_handler.py`), la generación de reportes (`reports.py`) y crear el formato de las tareas (`Tarea.py`).

Explicación de código importante

- Función recursiva `getTareasCompletadas`: Esta función recorre la lista de tareas y obtiene las tareas que han sido completadas para poder realizar un reporte. Si la clave `str(item)` no existe en el diccionario de tareas lanzará un error (`KeyError`) y devolverá el valor 0 deteniendo la recursividad. Mientras haya tareas a comprobar, el programa comprobará que el campo completado sea verdadero y devolverá 1, si no devuelve 0. Al finalizar el método realiza una suma de los valores devolviendo el total de tareas completadas.
- Función `to_dict`: Esta función en el archivo `Tareas` permite serializar las tareas creadas anteriormente para poder ser guardadas en el archivo JSON.
- Función `from_dict`: Se encuentra en el archivo `utils`, y permite serializar los datos obtenidos del archivo JSON al objeto `Tarea` y guardarlos en el diccionario.

Uso de IA

Durante la realización de este proyecto he utilizado inteligencia artificial como herramienta de apoyo para la resolución de errores no comunes. El código final ha sido elaborado por el autor.

Conclusión

Este proyecto me ha permitido desarrollar una aplicación funcional en el lenguaje Python para gestionar tareas haciendo uso de mis conocimientos de Programación Orientada a Objetos, manejo de archivos y la persistencia de datos en archivos JSON.

Como posibles mejoras futuras, sería recomendable la creación de una interfaz de usuario para hacer la aplicación más cómoda para el usuario.

Otra futura mejora podría ser la internalización de la aplicación mediante el añadido de diferentes lenguajes para que usuarios de diferentes países puedan hacer uso de la aplicación.

A parte de lo anterior, se podría en un futuro conectar la aplicación con diferentes API's y modificar el sistema de guardado a una base de datos online para poder ejecutar la aplicación en diferentes dispositivos.